

Closing a drawer now writes to your books

What changed: last slice built the cash drawer entries and showed them to you on screen. They were a worked example — real arithmetic, but nothing was posted. This slice connects them. When a manager reconciles a drawer, the entry goes into the ledger.

The chain, link by link

A cashier closes a drawer and counts it *blind*. That count is stored, and nothing posts yet — at blind close, over/short is genuinely unknown, and inventing a figure would be a guess. Later a manager supplies what the register says it sold and hits **Reconcile**. That is the first moment over/short is a *fact*, so that is when the entry posts: takings move to Undeposited Funds, the till is relieved, and the difference lands in `50920 Cash Over / (Short)`. The manager sees a sentence saying exactly what the ledger did, and the same answer goes to the audit log, so it survives the next click.

The float stays in the drawer. Only the day's takings go to the safe, which matches how you actually run the registers.

Tips are not touched. Tip money is employee money. If it were swept into Undeposited Funds along with the takings, the drawer would balance perfectly and your books would quietly claim the business earned the tips. Three tests guard this, including one that reads the source and proves the posting code never looks at the tips field at all.

The mistake this slice made, and how it got caught

The posting code needs three figures: what the drawer opened with, what it counted, and what it should have been. It refused to post when the counted or expected figure was missing — but for the *opening float* it quietly used zero.

That is not a rounding error. The opening float gets subtracted out to work back to the day's cash sales. Every register in your building is set to a \$167.50 float. Substitute zero and the entry claims that register took an extra \$167.50 — and the difference lands in the one account whose entire job is telling you whether a cashier can be trusted with money. A report that manufactures shortages is worse than no report.

No test caught it, because every test drawer had a float, just like every real one does. It was found by deliberately breaking the code to see whether the tests would notice — they didn't, and that silence was the evidence. It is written up as **D-74**. The fix removes the fallback entirely rather than just testing around it, and a companion test pins the other half: a drawer genuinely opened with no float is real and still posts.

The same exercise found that one of my own checks had been passing on luck. It searched the whole file for a line that appears five times, so it kept finding the wrong one. It has been narrowed, with a second test proving the narrowing is necessary. Thirty-six deliberate breaks, thirty-six caught.

What moved on the scoreboard, including one number going down

Two ledger rows — the till close and over/short — moved from *unreachable* to *reachable*. Over/short is now the third row in the whole system that is proven at every layer. The backlog count went from nine down to seven.

Compare that to last slice, where the same count went **up**, from six to nine, because building half a loop is honest progress that looks like regression. The safety check that refused to let me call last slice's work "reachable" is the same check that allowed this move without complaint. The bar didn't drop; the code cleared it.

What is still open, on purpose

Opening a drawer still posts nothing. When a float comes out of the safe, the system records the amount counted into the drawer but not *where it came from*. Posting it would assert the vault was drawn down when it may not have been — a guess about real money. It stays unwired and listed as outstanding rather than papered over.

Matching a safe drop to the actual bank deposit is also still open. Cash counted is not cash banked, and that link needs the Plaid feed.

Gates: 500 test files, 12,733 tests, all green. TypeScript, lint, pure self-tests and the quote verifier all clean. Nothing here was checked against your live database — no credentials exist in this environment.